



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

Data Structures & Algorithms in Python

Stacks and Queues – Lab Answers

Dr. Owen Noel Newton Fernando

College of Engineering

School of Computer Science and Engineering

Q01

1. Write a function `removeUntil()` that removes all values from a stack of integers until but not including the first occurrence of a given value. The function accepts two parameters: a reference to the stack and the value to stop at. The function definition is as follows:

```
def removeUntil(stack, value):
```

Let's go through the answer step-by-step

1. Class Definitions

```
class ListNode:
    def __init__(self, item):
        self.item = item
        self.next = None
```

This defines a node in a singly linked list.

In a linked list, each node stores:

- **Data** (the actual value)
- **A link** (pointer) to the next node

Q01 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class LinkedList:
    def __init__(self):
        self.size = 0
        self.head = None
        self.tail = None
```

This represents the entire linked list structure, which is basically a collection of connected ListNode objects.

self.size: Keeps track of how many nodes are currently in the list.

self.head: Reference to the first node in the list (the "entry point").

self.tail: Reference to the last node in the list.

Q01 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class Stack:  
    def __init__(self):  
        self.ll = LinkedList()
```

- Every stack here is implemented using a **linked list**.
- **self.ll** creates an empty linked list inside the stack to store elements.

Q01 - Answer

2. Functions

is_empty(self)

Checks whether the stack has no elements.

Since our stack is backed by a LinkedList (self.ll), the linked list keeps track of its size using **self.ll.size**.

If the size is 0, the stack is **empty** → **return True**.

Otherwise → **return False**.



```
def is_empty(self):  
    return self.ll.size == 0
```

Q01 - Answer

2. Functions

[push\(self, item\)](#) , [pop\(self\)](#) and [peek\(self\)](#)

Add a new element to the top of the stack.

It calls **insert_node(0, item)**, which inserts the item at index 0 of the linked list (the head).



```
def push(self, item):  
    self.ll.insert_node(0, item)
```

```
def pop(self):  
    if self.is_empty():  
        return None  
  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

```
def peek(self):  
    if self.is_empty():  
        return None  
  
    return self.ll.head.item
```

Q01 - Answer

2. Functions

push(self, item) , pop(self) and peek(self)

Remove and return the top element of the stack.

First, check if stack is **empty** (is_empty).

If not:

- Store the value of the **head** (**self.ll.head.item**).
- **Remove** the **head** node from the linked list (**remove_node(0)**).
- **Return** the stored **value**.



```
def push(self, item):  
    self.ll.insert_node(0, item)
```

```
def pop(self):  
    if self.is_empty():  
        return None  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

```
def peek(self):  
    if self.is_empty():  
        return None  
    return self.ll.head.item
```

Q01 - Answer

2. Functions

[push\(self, item\)](#) , [pop\(self\)](#) and [peek\(self\)](#)

Look at the top element without removing it.

- If **empty**, return **None**.
- Otherwise, **return** the **value** stored in head.



```
def push(self, item):  
    self.ll.insert_node(0, item)
```

```
def pop(self):  
    if self.is_empty():  
        return None  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

```
def peek(self):  
    if self.is_empty():  
        return None  
    return self.ll.head.item
```

Q01 - Answer

2. Functions

Answer: remove_until(stack, value)

Loop while the stack is not empty



```
def remove_until(stack, value):  
    while not stack.is_empty():  
        if stack.peek() != value:  
            stack.pop()  
        else:  
            break
```

Q01 - Answer

2. Functions

Answer: remove_until(stack, value)

Check the top element

- **stack.peek()** looks at the item on top without removing it
- We compare it with the **target value**.
- If **top $\neq$ value** $\rightarrow$ **remove it**
- If the top element is not equal to the value, **we remove it with pop()**.



```
def remove_until(stack, value):  
    while not stack.is_empty():  
        if stack.peek() != value:  
            stack.pop()  
        else:  
            break
```

Q01 - Answer

2. Functions

Answer: remove_until(stack, value)

If top == value → stop

- Once the top element matches the value, we stop removing (because the question says: **remove until**, but not including).

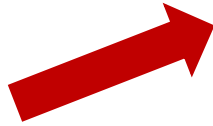


```
def remove_until(stack, value):  
    while not stack.is_empty():  
        if stack.peek() != value:  
            stack.pop()  
        else:  
            break
```

Q01 - Answer

Testing our functions

An empty stack is created.



```
if __name__ == "__main__":  
    stack = Stack()  
    items = [1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1]  
    for item in items:  
        stack.push(item)  
  
    print("Original Stack:")  
    stack.ll.print_list()  
    remove_until(stack, 5)  
    print("Stack after remove_until:")  
    stack.ll.print_list()
```

Q01 - Answer

Testing our functions

Push them one by one

Stacks are LIFO (Last-In, First-Out) → the last pushed item is on top.



```
if __name__ == "__main__":  
    stack = Stack()  
    items = [1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1]  
    for item in items:  
        stack.push(item)  
  
    print("Original Stack:")  
    stack.ll.print_list()  
    remove_until(stack, 5)  
    print("Stack after remove_until:")  
    stack.ll.print_list()
```

Q01 - Answer

Testing our functions

```
if __name__ == "__main__":  
    stack = Stack()  
    items = [1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1]  
    for item in items:  
        stack.push(item)
```

Print original stack



```
print("Original Stack:")  
stack.ll.print_list()  
remove_until(stack, 5)  
print("Stack after remove_until:")  
stack.ll.print_list()
```

Q01 - Answer

Testing our functions

Call `remove_until(stack, 5)`

This removes items from the top until it finds a 5.

Steps:

- Top = 1 → not 5 → pop it
- Top = 2 → not 5 → pop it
- Top = 3 → not 5 → pop it
- Top = 4 → not 5 → pop it
- Top = 5 → match! → stop

```
if __name__ == "__main__":  
    stack = Stack()  
    items = [1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1]  
    for item in items:  
        stack.push(item)  
  
    print("Original Stack:")  
    stack.ll.print_list()  
    remove_until(stack, 5)  
    print("Stack after remove_until:")  
    stack.ll.print_list()
```



Q01 - Answer

Testing our functions

```
if __name__ == "__main__":  
    stack = Stack()  
    items = [1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1]  
    for item in items:  
        stack.push(item)  
  
    print("Original Stack:")  
    stack.ll.print_list()  
    remove_until(stack, 5)  
    print("Stack after remove_until:")  
    stack.ll.print_list()
```

Print updated stack

Output: 5 6 5 4 3 2 1



Write a recursive function `recursiveReverse()` that reverses the order of items stored in a queue of integers. The function accepts a single parameter: a reference to the queue. The function definition is as follows:

```
def recursiveReverse(queue) :
```

Q02 - Answer

Let's go through the answer step-by-step

1. Class Definitions

```
class Queue:  
    def __init__(self):  
        self.ll = LinkedList()
```

This defines the Queue class that models a queue data structure.

self.ll is an instance of the LinkedList class.

Q02 - Answer

2. Functions

is_empty(self)

This method checks whether the queue has any elements or not.

- If self.ll.size is 0, the linked list is empty, meaning the queue has no elements



```
def is_empty(self):  
    return self.ll.size == 0
```

Q02 - Answer

2. Functions

enqueue(self, item)

Adds an element to the end of the queue.

- **self.ll.insert_node(self.ll.size, item)**
 - **self.ll** is the linked list storing the queue elements.
 - **insert_node(index, value)** inserts a new node at a given position.
 - **self.ll.size** is the current number of elements, so inserting at this index appends the new **item** at the end of the list, which is the rear of the queue.



```
def enqueue(self, item):  
    self.ll.insert_node(self.ll.size, item)
```

```
def dequeue(self):  
    if self.is_empty():  
        return None  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

Q02 - Answer

2. Functions

dequeue(self)

Removes and returns the front element of the queue



```
def enqueue(self, item):  
    self.ll.insert_node(self.ll.size, item)
```

```
def dequeue(self):  
    if self.is_empty():  
        return None  
  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

Q02 - Answer

2. Functions

dequeue(self)

if self.is_empty(): return None

Checks if the queue is empty; if yes, there's nothing to remove, so returns None.



```
def enqueue(self, item):  
    self.ll.insert_node(self.ll.size, item)
```

```
def dequeue(self):  
    if self.is_empty():  
        return None  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

Q02 - Answer

2. Functions

dequeue(self)

self.ll.head points to the first node of the linked list, which represents the front of the queue.

Store its value in **item** before removing it



```
def enqueue(self, item):  
    self.ll.insert_node(self.ll.size, item)
```

```
def dequeue(self):  
    if self.is_empty():  
        return None  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

Q02 - Answer

2. Functions

dequeue(self)

Removes the first node in the linked list, effectively removing the front element from the queue.



```
def enqueue(self, item):  
    self.ll.insert_node(self.ll.size, item)  
  
def dequeue(self):  
    if self.is_empty():  
        return None  
    item = self.ll.head.item  
    self.ll.remove_node(0)  
    return item
```

Q02 - Answer

2. Functions

Answer: recursive reverse(queue)

Base case

- The **recursion stops** when the queue is **empty**.
- This ensures we don't try to dequeue from an empty queue.



```
def recursive_reverse(queue):  
    if queue.is_empty():  
        return  
    item = queue.dequeue()  
    recursive_reverse(queue)  
    queue.enqueue(item)
```

Q02 - Answer

2. Functions

Answer: recursive reverse(queue)

Dequeue the front element

- Removes the front element of the queue and stores it in **item**.
- After this, the remaining queue has one less element.



```
def recursive_reverse(queue):  
    if queue.is_empty():  
        return  
    item = queue.dequeue()  
    recursive_reverse(queue)  
    queue.enqueue(item)
```

Q02 - Answer

2. Functions

Answer: recursive_reverse(queue)

Recursive call

- The function now calls itself on the smaller queue (without the first element).
- This continues until the queue becomes empty (base case).



```
def recursive_reverse(queue):  
    if queue.is_empty():  
        return  
    item = queue.dequeue()  
    recursive_reverse(queue)  
    queue.enqueue(item)
```

Q02 - Answer

2. Functions

Answer: recursive reverse(queue)

Re-enqueue the removed item

- Once the recursion starts returning, each removed item is added back to the end of the queue.
- Since the first removed item goes last, the queue is reversed.



```
def recursive_reverse(queue):  
    if queue.is_empty():  
        return  
    item = queue.dequeue()  
    recursive_reverse(queue)  
    queue.enqueue(item)
```

Q02 - Answer

2. Functions

Answer: recursive reverse(queue)

Example

Original queue: 1 2 3 4

Dequeue 1, recursive call with 2 3 4

Dequeue 2, recursive call with 3 4

Dequeue 3, recursive call with 4

Dequeue 4, recursive call with empty queue → base case returns

Then on returning:

Enqueue 4 → queue: 4

Enqueue 3 → queue: 4 3

Enqueue 2 → queue: 4 3 2

Enqueue 1 → queue: 4 3 2 1

Q03

Write a function `palindrome()` that determines whether a given string is a palindrome. The function accepts a single parameter: the word (a string). The function should return `True` if the string is a palindrome and `False` otherwise. The function should ignore whitespace, case, and punctuation. The function definition is as follows:

```
def palindrome(word) :
```

Q03 - Answer

Now we will go through the palindrome(word) function

Initialize Data Structures

Stack (LIFO): Last In, First Out. We'll use this to compare characters from the end of the word.

Queue (FIFO): First In, First Out. We'll use this to compare characters from the start of the word.



```
def palindrome(word):  
    stack = Stack()  
    queue = Queue()  
  
    word = word.upper()  
    for char in word:  
        if char.isalnum():  
            stack.push(char)  
            queue.enqueue(char)  
  
    while stack.ll.size > queue.ll.size:  
        stack.pop()  
  
    while not stack.is_empty():  
        if stack.pop() != queue.dequeue():  
            print("The string is not a palindrome")  
            return False  
  
    print("The string is a palindrome")  
    return True
```

Q03 - Answer

Now we will go through the palindrome(word) function

Normalize Input

Converts the word to uppercase so the comparison is case-insensitive.

Only considers alphanumeric characters (char.isalnum()), ignoring spaces and punctuation.

Each valid character is pushed to the stack and enqueued in the queue:

- **Stack:** Last character pushed → top of stack.
- **Queue:** First character enqueued → front of queue.



```
def palindrome(word):  
    stack = Stack()  
    queue = Queue()  
  
    word = word.upper()  
    for char in word:  
        if char.isalnum():  
            stack.push(char)  
            queue.enqueue(char)  
  
    while stack.ll.size > queue.ll.size:  
        stack.pop()  
  
    while not stack.is_empty():  
        if stack.pop() != queue.dequeue():  
            print("The string is not a palindrome")  
            return False  
  
    print("The string is a palindrome")  
    return True
```

Q03 - Answer

Now we will go through the palindrome(word) function

If the **stack** and **queue** somehow got out of sync (different sizes), this loop would remove extra elements from the stack to match the queue.

This is added as a “safety check” in case non-alphanumeric characters were filtered inconsistently.



```
def palindrome(word):
    stack = Stack()
    queue = Queue()

    word = word.upper()
    for char in word:
        if char.isalnum():
            stack.push(char)
            queue.enqueue(char)

    while stack.ll.size > queue.ll.size:
        stack.pop()

    while not stack.is_empty():
        if stack.pop() != queue.dequeue():
            print("The string is not a palindrome")
            return False

    print("The string is a palindrome")
    return True
```

Q03 - Answer

Now we will go through the palindrome(word) function

Compare Characters

Pop from stack → gets the last character of the word.

Dequeue from queue → gets the first character of the word.

Compare the two:

- If **different**, the word is **not a palindrome**, and the function returns **False**.
- **Otherwise**, continue until the stack is empty.



```
def palindrome(word):  
    stack = Stack()  
    queue = Queue()  
  
    word = word.upper()  
    for char in word:  
        if char.isalnum():  
            stack.push(char)  
            queue.enqueue(char)  
  
    while stack.ll.size > queue.ll.size:  
        stack.pop()  
  
    while not stack.is_empty():  
        if stack.pop() != queue.dequeue():  
            print("The string is not a palindrome")  
            return False  
  
    print("The string is a palindrome")  
    return True
```

Q03 - Answer

Now we will go through the palindrome(word) function

```
def palindrome(word):  
    stack = Stack()  
    queue = Queue()  
  
    word = word.upper()  
    for char in word:  
        if char.isalnum():  
            stack.push(char)  
            queue.enqueue(char)  
  
    while stack.ll.size > queue.ll.size:  
        stack.pop()  
  
    while not stack.is_empty():  
        if stack.pop() != queue.dequeue():  
            print("The string is not a palindrome")  
            return False  
  
    print("The string is a palindrome")  
    return True
```

If All Characters Match

If the loop **completes** without finding a **mismatch**, the word is a **palindrome**.



Q04

Write a function `balanced()` that determines if an expression comprised of the characters `()[]{}` is balanced. The function accepts a single parameter: the expression (a string). The function should return `True` if the expression is balanced and `False` otherwise.

The function definition is as follows:

```
def balanced(expression) :
```

Q04 - Answer

Now we will go through the balanced(expression) function

Initialize the stack



```
def balanced(expression):  
    stack = Stack()  
    matching = {'(': ')', '[': ']', '{': '}'  
  
    for char in expression:  
        if char in '([{':  
            stack.push(char)  
        elif char in ')]}':  
            if stack.isEmpty() or stack.pop()  
            != matching[char]:  
                return False  
  
    return stack.isEmpty()
```

Q04 - Answer

Now we will go through the balanced(expression) function

Matching dictionary

matching maps closing brackets to their corresponding opening brackets.



```
def balanced(expression):  
    stack = Stack()  
    matching = {')': '(', ']': '[', '}': '{'}  
  
    for char in expression:  
        if char in '([{':  
            stack.push(char)  
        elif char in ')]}':  
            if stack.isEmpty() or stack.pop()  
            != matching[char]:  
                return False  
  
    return stack.isEmpty()
```

Q04 - Answer

Now we will go through the balanced(expression) function

The loop examines every character in the input expression.



```
def balanced(expression):  
    stack = Stack()  
    matching = {'(': ')', '[': ']', '{': '}'  
    for char in expression:  
        if char in '([{':  
            stack.push(char)  
        elif char in ')]}':  
            if stack.isEmpty() or stack.pop()  
            != matching[char]:  
                return False  
    return stack.isEmpty()
```

Q04 - Answer

Now we will go through the balanced(expression) function

If the character is an opening bracket:
({

Push it onto the stack because we need a matching closing bracket later.



```
def balanced(expression):  
    stack = Stack()  
    matching = {')': '(', ']': '[', '}': '{'}  
  
    for char in expression:  
        if char in '([{':  
            stack.push(char)  
        elif char in ')]}':  
            if stack.isEmpty() or stack.pop()  
            != matching[char]:  
                return False  
  
    return stack.isEmpty()
```

Q04 - Answer

Now we will go through the balanced(expression) function

If the character is a closing bracket:)}]}

Check two conditions:

- `stack.isEmpty()` → If the stack is **empty**, there's no opening bracket for this closing bracket → **unbalanced**.
- `stack.pop() != matching[char]` → If the last opening bracket **does not match** the current closing bracket → **unbalanced**.

If either is true, the function returns **False** immediately.



```
def balanced(expression):  
    stack = Stack()  
    matching = {')': '(', ']': '[', '}': '{'}  
  
    for char in expression:  
        if char in '([{':  
            stack.push(char)  
        elif char in ')]}':  
            if stack.isEmpty() or stack.pop()  
            != matching[char]:  
                return False  
  
    return stack.isEmpty()
```

Q04 - Answer

Now we will go through the balanced(expression) function

Final check

After processing all characters:

- If the stack is empty → all opening brackets had matching closing brackets → **balanced** → returns **True**.
- If the stack is not empty → some opening brackets never closed → **unbalanced** → returns **False**.

```
def balanced(expression):  
    stack = Stack()  
    matching = {'(': ')', '[': ']', '{': '}'  
    for char in expression:  
        if char in '([{':  
            stack.push(char)  
        elif char in ')]}':  
            if stack.isEmpty() or stack.pop()  
               != matching[char]:  
                return False  
    return stack.isEmpty()
```

